

DATA VALIDATION ENGINEER · ONBOARDING PREP

Important MySQL Keywords

The keywords you will type and hear every day in source-to-target checks — not every reserved word in the manual.

Query skeleton · SELECT / JOIN / WHERE

Validation functions · COUNT, SUM, IS NULL

DDL, DML, constraints, window functions

25 words to know cold on day 1

Ready-to-run validation cheat sheet

Study sheet · Keep this next to Spoon / Jira tomorrow

How to use this sheet

For a Data Validation Engineer, “important MySQL keywords” means the ones you type and hear every day in source-to-target checks — not every reserved word in the MySQL grammar.

SQL is still your main weapon. Pentaho / PDI tells you how data moved. These keywords prove what landed.

1. Query skeleton — memorize this

Those eight words are the backbone of almost every validation query:

```
SELECT    -- columns / expressions
FROM      -- table
JOIN      -- other tables
WHERE     -- row filter (before grouping)
GROUP BY  -- collapse rows
HAVING    -- filter after grouping
ORDER BY  -- sort
LIMIT     -- cap rows
```

2. SELECT / read keywords

Keyword	Use
SELECT	Read data
FROM	Source table
AS	Alias (src.claims AS c)
DISTINCT	Unique rows/values
ALL	Opposite of DISTINCT (default)
WHERE	Filter rows
AND / OR / NOT	Combine conditions
IN / NOT IN	Match a list or subquery
BETWEEN	Inclusive range
LIKE / NOT LIKE	Pattern (% and _)
IS NULL / IS NOT NULL	Null checks — never use = NULL
EXISTS / NOT EXISTS	Subquery has rows
GROUP BY	Aggregate
HAVING	Filter groups (HAVING COUNT(*) > 1)
ORDER BY	Sort (ASC / DESC)
LIMIT	First N rows
OFFSET	Skip N rows
UNION	Combine, drop duplicates
UNION ALL	Combine, keep duplicates (faster, safer for counts)
CASE / WHEN / THEN / ELSE / END	If/else in SQL
CAST / CONVERT	Change type
IF()	Short if (MySQL-specific)

Keyword	Use
IFNULL() / COALESCE()	Null replacement
NULLIF()	Return NULL if equal

Duplicate check (you will write this constantly):

```
SELECT member_id, COUNT(*) AS cnt
FROM dw.claims_fact
GROUP BY member_id
HAVING COUNT(*) > 1;
```

3. JOIN keywords

Keyword	Meaning
JOIN / INNER JOIN	Only matching rows on both sides
LEFT JOIN / LEFT OUTER JOIN	All left + matches from right
RIGHT JOIN	All right + matches from left (rarely used)
CROSS JOIN	Cartesian product
ON	Join condition
USING	Join on same column name
NATURAL JOIN	Auto-join same names — avoid, dangerous

Orphan check (left has no match in right):

```
SELECT c.claim_id
FROM dw.claims_fact c
LEFT JOIN dw.member_dim m ON c.member_id = m.member_id
WHERE m.member_id IS NULL;
```

4. Comparison and operators

Keyword / operator	Meaning
= <> !=	Equal / not equal
< > <= >=	Compare
AND OR NOT	Logic
IN NOT IN	Set membership
BETWEEN	Inclusive range
LIKE	Pattern
REGEXP / RLIKE	Regex (MySQL)
IS	Used with NULL, TRUE, FALSE
DIV	Integer division
MOD / %	Remainder
+ - * /	Math
CONCAT()	String join (do not rely on unless PIPES_AS_CONCAT)

5. Aggregate functions (validation workhorses)

Keyword	Use
COUNT(*)	Row count
COUNT(col)	Non-null count
COUNT(DISTINCT col)	Unique values
SUM()	Totals (amounts)
AVG()	Average
MIN() / MAX()	Range / dates
GROUP_CONCAT()	Glue values in a group (MySQL)

Source vs target:

```
SELECT COUNT(*) FROM src.claims WHERE load_dt = '2026-08-22';
SELECT COUNT(*) FROM dw.claims_fact WHERE load_dt = '2026-08-22';
```

6. NULL, types, and conversion

Keyword	Use
NULL	Missing value
IS NULL	Test for missing
COALESCE(a, b)	First non-null
IFNULL(a, b)	MySQL: if a is null, b
CAST(x AS SIGNED)	Convert
CONVERT(x, CHAR)	Convert
SIGNED / UNSIGNED	Integer signedness
CHAR BINARY DATE	Cast targets
DATETIME DECIMAL	
TRUE / FALSE	Booleans (stored as 1/0)

Never: WHERE col = NULL

Always: WHERE col IS NULL

7. String functions you will hear

Keyword	Use
CONCAT() / CONCAT_WS()	Join strings
TRIM() / LTRIM() / RTRIM()	Strip spaces
UPPER() / LOWER()	Case
LENGTH() / CHAR_LENGTH()	Size
SUBSTRING() / SUBSTR()	Slice
LEFT() / RIGHT()	Ends of string
REPLACE()	Swap text
INSTR() / LOCATE()	Find position

Keyword	Use
LPAD() / RPAD()	Pad codes

Validation example: mapping says "trim + upper member_id":

```
SELECT COUNT(*)
FROM src.claims s
JOIN dw.claims_fact t ON UPPER(TRIM(s.member_id)) = t.member_id
WHERE s.member_id <> t.member_id; -- catch transform misses
```

8. Date / time keywords

Keyword	Use
DATE TIME DATETIME	Types
TIMESTAMP YEAR	
NOW() CURDATE()	Now
CURTIME()	
CURRENT_TIMESTAMP	
DATE() TIME() YEAR()	Extract
MONTH() DAY()	
DATE_FORMAT()	Display format
STR_TO_DATE()	Parse string to date
DATE_ADD() DATE_SUB()	Shift dates
INTERVAL	
DATEDIFF() TIMESTAMPDIFF()	Difference
EXTRACT()	Part of date
DAYOFWEEK() WEEKDAY()	Week

```
WHERE load_dt >= DATE_SUB(CURDATE(), INTERVAL 1 DAY)
```

INTERVAL is a real keyword: INTERVAL 7 DAY, INTERVAL 1 MONTH.

9. Window functions (MySQL 8 — very useful)

Keyword	Use
OVER	Window
PARTITION BY	Groups without collapsing rows
ORDER BY (inside OVER)	Rank order
ROW_NUMBER()	1, 2, 3... per partition
RANK() / DENSE_RANK()	Ranking
LAG() / LEAD()	Previous / next row
SUM() OVER()	Running total

Find duplicates and keep a winner:

```

SELECT *
FROM (
  SELECT *,
    ROW_NUMBER() OVER (PARTITION BY claim_id ORDER BY updated_at DESC) AS rn
  FROM dw.claims_fact
) x
WHERE rn > 1;

```

10. Subquery keywords

Keyword	Use
IN (SELECT ...)	Membership
EXISTS (SELECT ...)	Row exists
ANY / SOME / ALL	Compare to a set
Derived table	FROM (SELECT ...) AS t — must have an alias in MySQL

11. DML — change data (QA only, never Prod)

Keyword	Use
INSERT INTO VALUES	Add rows
INSERT ... SELECT	Insert from query
UPDATE SET	Change rows
DELETE FROM	Remove rows
REPLACE	Insert or overwrite (MySQL)
TRUNCATE	Empty table (DDL-like, hard to roll back)
ON DUPLICATE KEY UPDATE	Upsert

You validate these. You rarely run UPDATE / DELETE yourself on day 1.

12. DDL — structure

Keyword	Use
CREATE ALTER DROP	Schema changes
RENAME	
TABLE DATABASE SCHEMA	Objects
INDEX VIEW	
DATABASE / SCHEMA	Same idea in MySQL
USE	Switch database
SHOW	Inspect (SHOW TABLES, SHOW DATABASES)
DESCRIBE / DESC	Column list
EXPLAIN	How the query runs

Day-1 inspection:

```
SHOW DATABASES;
USE qa_dw;
SHOW TABLES;
DESCRIBE claims_fact;
SHOW CREATE TABLE claims_fact;
```

13. Constraints and table options

Keyword	Meaning
PRIMARY KEY	Unique row identity
FOREIGN KEY	Parent-child
REFERENCES	Points at parent table
UNIQUE	No duplicates
NOT NULL	Required
NULL	Allowed empty
DEFAULT	Fallback value
CHECK	Rule (MySQL 8.0.16+)
AUTO_INCREMENT	Surrogate key
CONSTRAINT	Named rule
INDEX / KEY	Speed lookups
ENGINE	InnoDB / MyISAM
CHARSET / COLLATE	Encoding / sort rules

14. Data types you will see on DESCRIBE

Numeric: TINYINT SMALLINT INT / INTEGER BIGINT DECIMAL / NUMERIC FLOAT DOUBLE BIT BOOLEAN / BOOL

String: CHAR VARCHAR TINYTEXT TEXT MEDIUMTEXT LONGTEXT ENUM SET BINARY VARBINARY BLOB

Date: DATE TIME DATETIME TIMESTAMP YEAR

JSON: JSON

For money / amounts, expect DECIMAL, not FLOAT.

15. Transactions (InnoDB)

Keyword	Use
START TRANSACTION / BEGIN	Start
COMMIT	Save
ROLLBACK	Undo
SAVEPOINT	Named undo point
LOCK / UNLOCK	Locks
FOR UPDATE	Lock rows in a SELECT

16. Control flow (procedures / scripts)

Keyword	Use
IF THEN ELSEIF ELSE END IF	Branch
CASE WHEN	Branch
LOOP REPEAT WHILE	Loops
LEAVE ITERATE	Break / continue
DECLARE	Variables in routines
SET	Assign (SET @x := 1)
CALL	Run a procedure
BEGIN END	Block

17. Access / safety words you will hear

Keyword	Meaning
GRANT / REVOKE	Permissions
PRIVILEGES	Rights
IDENTIFIED BY	Password (users)
READ / WRITE	Lock type
TEMPORARY	Temp table (session)
VIEW	Saved SELECT
WITH CHECK OPTION	View constraint

18. MySQL-specific words you will bump into

Keyword	Meaning
LIMIT	Not TOP (that is SQL Server)
AUTO_INCREMENT	Not IDENTITY / SEQUENCE
IFNULL	MySQL cousin of Oracle NVL
GROUP_CONCAT	MySQL-only aggregate
backticks ``	Quote names (`order` is reserved)
STRAIGHT_JOIN	Force join order (rare)
FORCE INDEX / USE INDEX / IGNORE INDEX	Hint
ON DUPLICATE KEY UPDATE	Upsert
REPLACE INTO	Delete + insert on key clash
INSERT IGNORE	Skip bad rows

19. Reserved words that bite you

Do not use these as table or column names without backticks:

```

SELECT ORDER GROUP INDEX KEY TABLE USER DATE TIME
STATUS TYPE NAME VALUE COUNT RANK RANGE LEFT RIGHT
MATCH DESC ASC DEFAULT CHECK CONSTRAINT DATABASE
SCHEMA PARTITION SYSTEM

```

If a table is named order, write:

```
SELECT * FROM `order`;
```

20. The 25 you should know cold tomorrow

- 1. SELECT 2. FROM 3. WHERE 4. AND / OR
- 5. JOIN / LEFT JOIN / ON
- 6. GROUP BY 7. HAVING 8. ORDER BY 9. LIMIT
- 10. DISTINCT 11. COUNT 12. SUM 13. MIN / MAX
- 14. IS NULL / IS NOT NULL
- 15. IN 16. BETWEEN 17. LIKE 18. EXISTS
- 19. CASE WHEN THEN ELSE END
- 20. CAST / CONVERT
- 21. COALESCE / IFNULL
- 22. UNION ALL
- 23. AS
- 24. DESCRIBE / SHOW
- 25. PARTITION BY / OVER / ROW_NUMBER (MySQL 8)

Validation cheat sheet — keywords in action

```

-- count
SELECT COUNT(*) FROM src.claims WHERE load_dt = '2026-08-22';

-- nulls
SELECT COUNT(*) FROM dw.claims_fact WHERE member_id IS NULL;

-- duplicates
SELECT claim_id, COUNT(*) FROM dw.claims_fact
GROUP BY claim_id HAVING COUNT(*) > 1;

-- orphans
SELECT c.claim_id
FROM dw.claims_fact c
LEFT JOIN dw.member_dim m ON c.member_id = m.member_id
WHERE m.member_id IS NULL;

-- transform (trim / case)
SELECT COUNT(*)
FROM src.claims s
JOIN dw.claims_fact t ON s.claim_id = t.claim_id
WHERE UPPER(TRIM(s.status)) <> t.status;

-- sample
SELECT * FROM dw.claims_fact ORDER BY claim_id LIMIT 20;

```

Rule of thumb: job success in Pentaho does not mean data is correct.

COUNT, duplicates, nulls, transforms, orphans, then a sample. Evidence goes on the Jira ticket.